CAHIER DES CHARGES

Plateforme open source d'orchestration d'agents IA de programmation

Nom de travail : Nexus AI | Version 0.2 — exigences MVP consolidées | 10 octobre 2026

Statut : projet de référence pour démarrer le développement ; hypothèses et arbitrages explicités.

1. Résumé exécutif

Application Desktop local-first sur Linux, Windows et macOS. Elle regroupe des agents de codage accessibles par CLI et des modèles accessibles par API, permet de construire plusieurs équipes par projet, de choisir un superviseur IA ou un pilotage manuel, d'exécuter des tâches en parallèle dans des espaces isolés et d'observer les actions, changements et limites d'utilisation. La mémoire commune est une base d'événements et de décisions persistante, non une conversation partagée aveuglément.

2. Analyse comparative vérifiée (fonctionnalités documentées, non audit de code exhaustif)

Produit	Forces documentées	Limites / risques relatifs	Ce que nous réutilisons comme principe
Hydra	Electron; PTY; Claude/Codex; superviseur MCP; sessions; usages.	Catalogue restreint; limite annoncée de 10 agents; gestion complexe des processus.	Grille multi-terminaux, statut, agent manager MCP.
Runner	Missions, rôles, flux d'événements; CLI de contrôle; usages; agent lead; service persistant.	Application alpha; pas de Linux Desktop annoncé; modèle de workflow opinionné.	Journal rejouable, protocole de messages, reprise.
Codey	Équipes; dispatcher facultatif; mémoire; graphes; roundtable; worktrees; MIT.	Desktop macOS principalement; complexité des graphes; plus de surface à maintenir.	Rôles réutilisables, mode conseil facultatif, mémoire versionnée.
Emdash	CLI autodétectés; worktrees; revues diffs; SSH; SQLite; multi-OS.	Orchestration du cerveau commun moins centrale dans la description; coût opérationnel des worktrees.	Découverte CLI, isolation Git, approbation des diffs.
Pivio	Agents côte à côte; multi-comptes, usage, kanban, pipelines; ergonomie.	Prise en charge initiale limitée aux CLIs indiqués; automatisation par cerveau commun à	Onboarding, vue agent, indicateurs et pipeline simple.

approfondir.

Conclusion : aucune unicité n'est acquise sur les fonctionnalités isolées. Le positionnement à tester est l'unification API+CLI, les politiques d'autonomie, la mémoire inter-agents vérifiable et les garde-fous sur les secrets.

3. Vision, public et périmètre

- Cible prioritaire : développeurs individuels, petites équipes, consultants et mainteneurs open source.
- Objet principal : coding agents. L'analyse documentaire ou la recherche ne sont autorisées que lorsqu'elles appuient un projet logiciel.
- Distribution : open source, local-first, sans compte central obligatoire.
- Langues d'interface : français et anglais en priorité, internationalisation dès le socle.
- Hors MVP : marketplace d'agents, hébergement universel one-click, mobile, collaboration multi-utilisateur cloud, débats multi-agents avancés.

4. Scénarios utilisateur

US-01 Démarrage

Au premier lancement, l'application détecte les CLI installés, leur chemin, leur version, la compatibilité et indique « prêt », « connexion requise », « non installé » ou « vérification impossible ». Elle ne lit jamais les secrets d'authentification.

US-02 Connexion

L'utilisateur ajoute des connexions API OpenAI, Anthropic, Google AI Studio, OpenRouter, endpoint local; ainsi que Claude Code, Codex, Gemini CLI, Hermes et autres CLI via adaptateurs.

US-03 Équipe

L'utilisateur crée des équipes et des profils: architecte, développeur, relecteur, QA, DevOps; chaque membre possède un connecteur, un modèle, un rôle, un budget et des permissions.

US-04 Manuel

L'utilisateur attribue lui-même les tâches et peut intervenir ou interrompre un agent.

US-05 Automatique

L'utilisateur choisit un cerveau principal; celui-ci prépare un DAG de tâches, choisit les agents disponibles selon capacités, budget, limites, dépendances et droits.

US-06 Parallèle

Deux agents travaillent dans deux worktrees; résultats et diffs sont inspectés; aucun merge automatique non autorisé.

US-07 Collaboration

Les agents échangent via une messagerie orchestrée. Ils peuvent demander un avis pair-à-pair sur des sujets non sensibles; toute nouvelle mission ou modification des permissions passe par le superviseur/politique.

US-08 Visibilité

L'utilisateur consulte étapes, sorties, commandes, fichiers modifiés, incidents, statut d'authentification, coût estimé et limites déclarées.

US-09 Sécurité

L'utilisateur définit droits par projet/agent; masque les secrets; autorise ou refuse réseau, déploiement, suppression et accès à des chemins sensibles.

US-10 Architecture et livraison

L'utilisateur génère un schéma d'architecture et, en phase ultérieure, reçoit un assistant pas-à-pas de déploiement.

5. Gouvernance et autonomie

Mode	Comportement	Politique par défaut
Manuel	L'humain assigne et valide les tâches.	Aucune délégation autonome.
Supervisé	IA planifie et délègue; actions à risque soumises à approbation.	Mode par défaut recommandé.
Automatique restreint	IA exécute les actions autorisées dans sandbox; limites de temps/coût.	Opt-in, périmètre vérifiable.
Automatique étendu	Exécution plus autonome avec droits explicites et journaux.	Jamais activé implicitement.

La communication agent-à-agent est permise par canal structuré, mais elle ne transfère ni privilèges ni secrets. Les ordres exécutoires nécessitent une autorisation conforme à la politique de la mission.

6. Architecture cible

Electron (main process, preload minimal, renderer React) → IPC typé → services locaux : ProjectManager / DiscoveryService / AdapterRegistry / Orchestrator / TaskQueue / EventBus / MemoryStore / GitWorkspaceManager / PolicyEngine / SecretsBroker / UsageMonitor / Observability. Un worker de processus séparé héberge les CLI et PTY. SQLite WAL stocke projets, équipes, jobs, événements, décisions et métriques. Les logs et artefacts sont stockés sur disque avec rétention configurable.

6.1 Contrat d'adaptateur

ProviderAdapter : probe() ; listModels() ; capabilities() ; authStatus() ; usageSnapshot() ; startSession() ; send() ; cancel() ; resume() ; subscribeEvents() ; dispose(). Les capacités sont optionnelles et décrites explicitement ; aucune supposition d'équivalence entre API et CLI.

Événements canoniques : AgentStarted, AgentOutput, ToolRequested, ToolApproved, FileChanged, UsageUpdated, AgentBlocked, AgentCompleted, AgentFailed, TaskAssigned, DecisionRecorded.

6.2 Mémoire commune

- Données structurées : exigences, plan, tâches, décisions avec auteur et justification, artefacts, dépendances, chemins Git, tests, résultats et problèmes.
- Chaque élément possède projet, horodatage, version, provenance, niveau de confiance et visibilité.
- Contexte rendu à chaque agent par requêtes ciblées; éviter de recopier tous les transcripts à chacun.
- Indexation sémantique optionnelle après MVP; SQLite + recherche textuelle en v0.1.
- Enregistrement événementiel immuable + états matérialisés; gestion de conflit optimiste pour écritures concurrentes.

6.3 Gestion parallèle du code

Chaque tâche modifiant du code dispose d'une branche/worktree dédiée. Le moteur identifie les dépendances, réserve les fichiers à haut risque lorsque nécessaire, exécute lint/types/tests puis présente diff, conflits et résultats. Le merge est une étape indépendante, auditée et réversible via Git.

7. Connecteurs et quotas

Famille	Connecteurs prioritaires	Usage/quota
CLI	Claude Code, Codex, Gemini CLI; Hermes en pilote si interface stable.	Collecte via commande officielle/CLI lorsque possible; sinon « inconnu ».
API	OpenAI, Anthropic, Google AI Studio, OpenRouter.	Compteurs de tokens/coûts à partir des réponses et tarifs configurés; quotas distants selon API disponible.
Local	Ollama / endpoint compatible.	Mesures locales; pas de notion uniforme de quota fournisseur.

Le système différencie tokens consommés, coût estimé, solde facturé, limite de session, quota hebdomadaire et taux d'utilisation. Aucune barre « % restant » n'est présentée si la donnée n'est pas officielle, suffisamment fiable et associée à une période claire. La collecte privilégie les API publiques et commandes officiellement prévues, jamais le scraping de cookies ou de jetons d'autres applications.

8. Sécurité et gestion des secrets — exigences bloquantes

- Clés API dans le coffre-fort du système (Secret Service/libsecret sous Linux, DPAPI/gestionnaire Windows, Keychain macOS), jamais dans SQLite en clair.
- Secrets des projets : .env et chemins configurés refusés par défaut aux outils de lecture et aux prompts; aucune ingestion dans l'index mémoire.
- SecretsBroker pour transmettre uniquement les variables autorisées à un processus qui en a besoin. Journaliser la décision, jamais la valeur.
- Désactivation des accès réseau et du montage de secrets dans les sandboxes lorsque compatibles.

- Redaction des sorties et scans de diffs/commits; blocage d'un commit contenant un secret détecté.
- Politique deny-by-default sur commandes destructrices, publication, push et déploiement.
- Attention : un CLI exécuté avec les droits OS de l'utilisateur peut lire les fichiers accessibles à ce compte; une protection forte exige un véritable isolement OS/conteneur/VM, non seulement un filtre UI.

9. Interface Desktop

- Onboarding : état des CLI et des API, tests de connexion, permissions demandées.
- Colonne projets et équipes; tableau central Kanban/tâches; panneaux d'agents et terminaux réels; onglet code/diff (Monaco Editor) et aperçu Git.
- Vue « cerveau » : plan, dépendances, délégations, décisions, budget et incidents.
- Vue « communication » : messages inter-agents avec provenance et possibilité de filtrer.
- Vue « usage » : statistiques observées, estimées ou indisponibles clairement différenciées.
- Paramètres : modes de gouvernance, sandbox, notifications sensibles, secrets, limites, rétention, export des données.

10. Architecture et déploiement assisté

MVP : l'agent peut analyser le dépôt, détecter framework, DB, variables requises (noms uniquement), dépendances et générer un diagramme Mermaid plus un plan de déploiement avec checklist. Version suivante : connecteurs GitHub Actions, Vercel, Netlify, Railway, VPS/SSH, Docker/Compose; dry-run et confirmation explicite avant toute publication, changement DNS ou commande distante. Ne jamais envoyer les fichiers .env à un prestataire sans consentement granulaire.

11. Exigences non fonctionnelles / critères de validation

- Plateformes : Ubuntu Linux cible de développement; livrables Windows, macOS et Linux testés en CI et sur machines réelles.
- Robustesse : redémarrage de l'application sans perte des tâches historiques; annulation propre et timeout des processus.
- Sécurité : pas de secrets dans les logs, commandes ou rapports; tests de régression de permissions; dépendances auditées.
- Explicabilité : chaque délégation automatique a une justification consultable (capacités, coût et disponibilité).
- Interopérabilité : un nouvel adaptateur ne doit pas imposer de modification du cœur de l'orchestrateur.
- Observabilité : sorties en streaming, horodatage, identifiant de tâche, audit et erreurs exploitables.
- Qualité : tests unitaires du scheduler/adapters/policies et tests E2E sur CLI factices avant intégration réelle.

12. Backlog priorisé

Lot	Priorité	Livrable démontrable

0. Socle	P0	Monorepo Electron/React/TS; SQLite; CI; logs et schéma de DB.
1. Détection	P0	Discovery Claude/Codex/Gemini + écran de compatibilité.
2. Exécution	P0	Un CLI piloté par PTY + statut + annulation.
3. Deux agents	P0	Deux sessions parallèles, événements, chat séparé.
4. Orchestration	P0	Équipes, tâches, manuel et superviseur IA, décision journalisée.
5. Isolation	P0	Git worktree par tâche, diff, tests, approbation merge.
6. Sécurité	P0	Permissions, secrets, environnement restreint et tests d'exfiltration.
7. API	P1	OpenRouter + un API direct, puis les autres.
8. Quotas	P1	Statuts fiables par provider, coûts estimés, « inconnu » si absent.
9. Schémas	P1	Diagrammes Mermaid et checklist de déploiement.
10. Extensions	P2	Hermes stable, autres CLI, déploiement guidé, conseil IA, mobile.

13. Premier cas d'essai et définition de « terminé »

Projet pilote : mini application de gestion de stock. Agent A crée la structure de l'API; agent B crée une UI minimale sur sa branche. Le cerveau commun enregistre les exigences et le contrat API. Chaque agent obtient son contexte ciblé; aucun ne voit les secrets de l'autre. L'humain visualise journaux, modifications et tests; l'outil propose l'intégration et attend validation. Fin du jalon si les deux agents réussissent, sans écrasement de fichiers, avec traçabilité et reproduction après redémarrage.

14. Architecture de dépôt recommandée

apps/desktop (Electron, React) • packages/core (types, événements) • packages/orchestrator (DAG, scheduler) • packages/adapters (cli, api) • packages/security (policy, secrets) • packages/storage (SQLite) • packages/git (worktrees) • packages/ui (composants) • tests/fixtures (CLI factices) • docs/ (ADR, schémas).

15. Arbitrages à revoir après le prototype

- Validation des licences des composants réutilisés, sans recopier du code concurrent incompatible.
- Choix technique de sandbox multi-OS : processus natif, conteneur ou VM selon plateforme et niveau de risque.
- Modes multi-comptes par fournisseur selon les méthodes d'authentification officiellement autorisées.
- Démonstration de gains mesurables : taux de réussite, temps de revue, conflits, coût, incidents, satisfaction utilisateurs.
- Choix futur de licence du projet : MIT ou Apache-2.0, avec politique de contribution et modèle économique ultérieur.

16. Références et périmètre de vérification

Hydra — https://github.com/jpdlr/hydra

Runner — https://github.com/yicheng47/runner

Codey — https://github.com/its-ahoh/codey

Emdash — https://github.com/generalaction/emdash

Pivio — https://pivio.app/

Claude Code – usage — https://support.claude.com/en/articles/14552983-models-usage-and-limits-in-claude-code

Gemini CLI – quotas — https://geminicli.com/docs/resources/quota-and-pricing/

Méthode : comparaison fondée sur README et documentation accessibles au 10 octobre 2026, non sur tests installés ni audit complet de code. Les limites des produits sont des écarts par rapport à notre besoin, pas des vulnérabilités confirmées.

17. Révision v0.2 — nouvelles exigences prioritaires

Statut : cette section complète et prévaut sur les mentions contradictoires des sections précédentes. Date : 10 octobre 2026. Vision inchangée : IDE agentique Desktop open source, API + CLI, orchestration manuelle et automatique, mémoire commune et collaboration parallèle.

17.1 Décisions actées

ID	Module	Priorité	Exigence
FR-VOICE	Saisie vocale	MVP	Bouton micro et raccourci, transcription éditable avant soumission; permissions microphone explicites; fallback texte; pas de stockage audio brut par défaut.
FR-GH	GitHub	MVP	Connexion via flux officiel/gh CLI, détection dépôt, branche, commit, push, création de PR en option; prévisualisation du diff; validation obligatoire avant push distant.
FR-DEPLOY	Vercel	MVP limité	Détection framework compatible, connexion via CLI/officielle, prévisualisation du plan et déploiement preview; déploiement production après confirmation; journal URL/logs.
FR-HOST	Autre hébergement	Phase 2	Architecture de plugins pour autres plateformes et VPS; dans le MVP, assistant d'hébergement et procédures guidées uniquement.
FR-SBOX	Sandbox CLI et secrets	MVP bloquant	Exécution d'agents non fiables en environnement isolé; permissions FS/réseau/processus; secret broker; accès aux secrets interdit par défaut.

FR-SESSION	Sessions persistantes	MVP bloquant	Conversations, tâches, événements, fichiers, décisions et identifiants de reprise durables; restauration après fermeture complète/redémarrage.
FR-LICENSE	Licence	Décision	Code source du noyau sous Apache-2.0; stratégie de monétisation par cloud hébergé, support, fonctionnalités entreprises distinctes et marque.
FR-QUOTA	Consommation	MVP progressif	Métriques obtenues exclusivement via voies documentées; indiquer source, horodatage, statut confirmé/estimé/indisponible; jamais de scraping fragile par défaut.

17.2 Saisie vocale (MVP)

- L'utilisateur lance l'enregistrement par clic ou raccourci clavier, voit un indicateur d'enregistrement, peut arrêter/annuler et relire la transcription.
- Le moteur de reconnaissance doit être interchangeable : local hors ligne en priorité si disponible, et fournisseur distant uniquement après consentement avec explication des coûts et transferts.
- La transcription ne devient jamais automatiquement une instruction d'exécution : édition et confirmation utilisateur avant dispatch; gestion des langues (français prioritaire, anglais).
- Critères : permission micro refusée => texte toujours disponible; audio non conservé par défaut; message explicite en cas de modèle STT absent.

17.3 GitHub : flux de publication (MVP)

- Onboarding : détecter git et gh; proposer une authentification officielle sans lire ni afficher les jetons; possibilité de choisir compte/dépôt.
- Détecter l'état Git et les modifications; suggérer nouvelle branche; afficher clairement fichiers, diff, commit et destination du push.
- Exiger approbation explicite pour tout push, force-push, suppression de branche, changement de branche protégée ou publication; force-push bloqué par défaut.
- Pousser vers le remote choisi, remonter l'URL du commit ou de la pull request, consigner l'événement; aucun secret ne doit être inclus dans commits/logs.
- Échec réseau/auth/permissions : conserver état et proposer reprise idempotente, sans répéter aveuglément les opérations.

17.4 Vercel et hébergement (MVP limité)

- Analyser le projet (framework, script build, runtime, domaine éventuel), produire un rapport de compatibilité et un plan de déploiement.
- Connexion officielle via Vercel CLI et authentification contrôlée; l'application n'exporte ni n'affiche les valeurs des secrets.
- Déploiement « preview » autorisé sur confirmation, journalisant URL, statut et logs expurgés; production exige une confirmation distincte explicite.
- Indiquer coûts/limites éventuels sans les inventer; prévoir consentement pour l'utilisation de ressources cloud.
- Hébergement VPS/Docker/managé : recommandations et diagrammes dans le MVP; automatisation fournisseur par fournisseur après stabilisation.

17.5 Isolation : deux frontières de sécurité distinctes

Ne pas confondre sandbox Electron des fenêtres graphiques avec sandbox des commandes exécutées par les agents. Le renderer doit avoir nodeIntegration=false, contextIsolation=true, sandbox=true, IPC validé, CSP restrictive. Les agents CLI nécessitent une frontière OS ou conteneur indépendante.

- Linux (plateforme de développement et MVP validé) : agent exécuté dans conteneur rootless ou isolation OS équivalente, utilisateur non privilégié, volumes projet à autorisations limitées, /home réel et clés personnelles non montés.
- Accès réseau fermé par défaut pour les commandes non approuvées, puis liste de destinations autorisées selon le connecteur et les besoins; connexions API/CLI sortantes explicitement configurées.
- Mode « automatisé », « notification » ou « strict » : les préférences modifient la politique d'approbation, mais ne suppriment jamais l'isolation minimale, l'audit et la protection des secrets.
- Windows et macOS : architecture prévue; versions distribuées seulement après tests réels de leur backend sandbox propre. Ne pas qualifier le sandbox de garanti sur OS non vérifié.
- Secret Guard : coffre OS, échange de jetons par connecteur/broker, filtrage des logs, refus d'accès direct aux .env/SSH/auth; déclarer les limites si un outil requiert des secrets en processus.
- Tests offensifs obligatoires : lecture ~/.ssh, fuite .env, exfiltration HTTP, accès hors workspace, escalade via symlinks, injection dans prompts/outils, tentative d'accès au socket Docker.

17.6 Persistance et reprise de session : contrat

- SQLite local en WAL avec transactions, migrations versionnées, politiques de rétention et export/suppression des données; contenu du projet jamais synchronisé vers un serveur Nexus à l'insu de l'utilisateur.
- Entités minimales : Project, Team, AgentProfile, Connection, Conversation, Message, Task, Run, Event, Artifact, Approval, GitOperation, Deployment et SecretReference.
- Chaque exécution dispose d'un ID stable, états queued/running/waiting_approval/paused/completed/failed/interrupted; journal append-only horodaté.
- À la fermeture : enregistrer un checkpoint (tâches, worktrees, sessions CLI si reprise supportée, dernier événement) et marquer clairement toute exécution interrompue.
- Au redémarrage : ouvrir le même projet et afficher fil de discussion, décisions, modifications Git et tâches en cours/à reprendre; si le CLI ne permet pas de reprendre une session native, reconstruire le contexte et signaler la reprise logique, jamais prétendre restaurer un processus mort.

- Critère d'acceptation : fermer totalement Electron, redémarrer la machine, ouvrir le projet et retrouver conversations, files de tâches, logs et résultats sans corruption ni doublons.

17.7 Licence et plan de monétisation

Recommandation retenue : Apache License 2.0 pour le cœur publié. Autorise usage, modification, redistribution et commercialisation par nous comme par des tiers; inclut des dispositions de brevets et des obligations de conservation des avis. Elle NE réserve PAS les droits d'exploitation commerciale à l'auteur.

Monétisation conseillée : hébergement managé, stockage et synchronisation multi-machines, équipes et gouvernance d'entreprise, support/SLA, intégrations administrées, sécurité avancée et audit hébergé. Garder clairement séparés le dépôt open source et tout module propriétaire futur; contrôler les licences des dépendances et contributions (CLA/DCO à examiner).

Avant publication publique : consulter un juriste en propriété intellectuelle sur politique de contributions, marques, droits d'auteur et compatibilité de licences. Nom « Nexus AI » provisoire, vérifier disponibilité de marque et de domaine.

17.8 Plan de livraison et critères de blocage

Lot A — base solide

Electron/React, SQLite et migrations, éditeur, découverte CLI, connecteurs Claude Code/Codex (si présents), permissions, journal d'événements et redémarrage fiable.

Lot B — cœur multi-agents

Dispatch manuel et superviseur IA, contrats d'adaptateurs, worktrees, messages inter-agents, budgets, interruption/reprise et tests d'intégration.

Lot C — productivité

Saisie vocale, connexion GitHub et push avec confirmation, assistant d'architecture Mermaid, déploiement preview Vercel et déploiement production avec confirmation.

Gate de publication

Ne pas publier comme MVP prêt à l'emploi tant que persistance, sandbox Linux, secrets, GitHub et Vercel ne passent pas les tests d'acceptation. Les versions Windows/macOS nécessitent validation séparée des backends de sandbox.

17.9 Références additionnelles vérifiées

- https://www.apache.org/licenses/LICENSE-2.0.html
- https://opensource.org/faq
- https://cli.github.com/manual/gh_auth_login
- https://vercel.com/docs/projects/deploy-from-cli
- https://www.electronjs.org/docs/latest/tutorial/security/
- https://www.electronjs.org/docs/latest/tutorial/sandbox